

Sistema de Control de Asistencia y Gestión de Turnos

ETAPA 3 IMPLEMENTACIÓN FINAL
AVANCE 1 — FORO DE DISCUSIÓN 1

INVESTIGACIÓN Y APLICACIÓN DE LIBRERÍAS JAVASCRIPT EN EL SISTEMA TIMESHIFT

Integrantes:

Boris Armando Campos Lopez — CL253217

Diego Humberto Marquez Valencia — MV262624

Marcos Abdulio López Alvarado — LA211485

Alan Enrique Ventura Hernández — VH262707

Desarrollo de Aplicaciones Web con Software Interpretados en el Cliente (DAW901)

Octubre de 2026

1. INTRODUCCIÓN

La digitalización del control de asistencia permite reducir tareas manuales y consultar con mayor claridad los períodos de trabajo, recesos y salidas. El proyecto TimeShift se desarrolla para atender esa necesidad en la Oficina Jurídica Jovita Alvarado mediante un portal para empleados, un panel administrativo, la gestión del personal y un módulo de reportes.

En el Avance 1 de la Etapa 3 se incorporaron librerías JavaScript para ofrecer visualizaciones dinámicas, mensajes de confirmación y exportación de información. Esta investigación explica los fundamentos técnicos de Chart.js, SweetAlert2, SheetJS y jsPDF con el complemento jsPDF-AutoTable, identifica su implementación en el código del proyecto y justifica por qué se eligieron frente a otras soluciones. La revisión distingue las funciones existentes del prototipo de las integraciones con React y APIs externas que fueron propuestas en la primera etapa, pero todavía no pertenecen a este avance.

2. CONTEXTO DEL PROYECTO Y NECESIDAD

2.1. Organización y problema identificado

La Oficina Jurídica Jovita Alvarado es una organización de servicios legales descrita en la Fase 1 como un equipo de nueve personas. El diagnóstico inicial identificó dificultades para seguir entradas, salidas y recesos, además de poca disponibilidad de métricas visuales para gerencia (Equipo TimeShift, 2026).

2.2. Alcance real del avance

En la versión revisada del repositorio, TimeShift utiliza HTML5, CSS, JavaScript del lado del cliente y almacenamiento local del navegador. Los estados operativos actuales son Disponible, Lunch, Break y Salida. El prototipo incluye dos tipos de acceso, portal de empleado, gestión de registros, dashboard-v2 y reportes. Chart.js y las librerías de exportación se cargan desde CDN; todavía no existe una base de datos compartida entre distintas computadoras.

La Fase 1 proponía una futura arquitectura React, funciones de puntualidad y servicios externos. Por rigor documental, esas propuestas no se presentan aquí como funcionalidades ya terminadas.

3. OBJETIVOS

3.1. Objetivo general

Analizar y justificar las librerías JavaScript aplicadas en TimeShift para mejorar la visualización de información, la interacción con el usuario y la generación de reportes del sistema.

3.2. Objetivos específicos

- Describir las características técnicas y ventajas de las librerías seleccionadas.
- Relacionar cada librería con archivos, funciones y flujos comprobables del proyecto.

- Explicar las decisiones de implementación y las limitaciones de esta versión.
- Evaluar cómo las mejoras satisfacen los requisitos del Foro 1 y proponer recomendaciones para etapas posteriores.

4. INVESTIGACIÓN TÉCNICA DE LAS LIBRERÍAS

4.1. Chart.js: representación de indicadores

Chart.js es una librería para construir gráficos mediante Canvas de HTML. Permite configurar etiquetas, conjuntos de datos, ejes y opciones de visualización; además admite actualizar un gráfico existente al cambiar sus valores mediante el método update() (Chart.js, s. f.). Esta característica resulta conveniente cuando se necesita mostrar la distribución de empleados por estado sin reconstruir manualmente los elementos del gráfico.

En dashboard-v2/app.js se utiliza renderAuxChart(employees), que calcula las cantidades de los cuatro AUX y crea una gráfica de barras con new Chart(canvas, ...). Si el objeto de la gráfica ya existe, su dataset se modifica y se ejecuta auxChart.update(). En reportes.html, drawChart(S) representa minutos agrupados por estado utilizando datos normalizados del historial.

```
const counts = stateIds.map(id => countEmployeesByState(employees, id));
auxChart.data.datasets[0].data = counts;
auxChart.update();
```

Decisión técnica: utilizar la misma fuente de datos que alimenta las tarjetas KPI evita contradicciones entre indicadores y gráfico. Un aspecto a vigilar es que los datos guardados en localStorage no constituyen monitoreo multiusuario entre dispositivos.

4.2. SweetAlert2: diálogo y retroalimentación

SweetAlert2 proporciona diálogos personalizables para confirmar operaciones, informar errores y mostrar notificaciones breves. En TimeShift, js/gestion-datos.js detecta la disponibilidad de Swal, configura notificaciones tipo toast y utiliza cuadros de confirmación para acciones administrativas, manteniendo un mecanismo alternativo mediante confirm() si la librería no carga (SweetAlert2, s. f.).

La decisión de emplear mensajes diferenciados antes y después de eliminar o editar registros reduce la ambigüedad de las acciones. Las validaciones de formularios siguen siendo parte de la lógica JavaScript del sistema; SweetAlert2 presenta la retroalimentación, pero no sustituye la validación de datos.

```
const result = await Swal.fire({
  title: "Confirmar operación",
  showCancelButton: true
});
```

4.3. SheetJS: exportación de hojas de cálculo

SheetJS permite construir hojas y libros XLSX desde arreglos de datos en JavaScript. Su API incluye XLSX.utils.aoa_to_sheet() para transformar arreglos bidimensionales en hojas, XLSX.utils.book_append_sheet() para unirlas al libro y XLSX.writeFile() para descargar el archivo en el navegador (SheetJS, s. f.).

En reportes.html, exportXLSX() crea tres hojas: “Registros AUX”, “Resumen por estado” y “Filtros”. Así, el usuario recibe los datos detallados y el contexto de la consulta en un mismo archivo. Esta estructura facilita la revisión posterior y reduce el riesgo de interpretar cifras fuera del rango filtrado.

```
const wb = XLSX.utils.book_new();
const ws = XLSX.utils.aoa_to_sheet([HEAD, ...rowsOut()]);
XLSX.utils.book_append_sheet(wb, ws, 'Registros AUX');
XLSX.writeFile(wb, `reporte_aux_${stamp()}.xlsx`);
```

4.4. jsPDF y jsPDF-AutoTable: documentos exportables

jsPDF construye archivos PDF desde JavaScript. El complemento jsPDF-AutoTable agrega tablas con encabezados y filas paginables para informes tabulares (Parallax, s. f.; simonbengtsson, s. f.). TimeShift utiliza ambas herramientas en la función buildPDF(m) de reportes.html, con página tamaño carta, secciones descriptivas, estadísticas y tablas de períodos AUX.

Este enfoque permite entregar un reporte legible sin convertir manualmente una tabla HTML. La exportación PDF operativa no debe confundirse con este documento de investigación: son productos distintos, aunque ambos pueden incluir textos, tablas y referencias.

```
const { jsPDF } = window.jspdf;
const doc = new jsPDF({ unit: 'pt', format: 'letter' });
doc.autoTable({ head: [['Estado AUX','Registros']], body: rows });
```

5. IMPLEMENTACIÓN DE LAS LIBRERÍAS EN TIMESHIFT

Librería / tecnología	Archivo y función	Aporte verificable
Chart.js	dashboard-v2/app.js · renderAuxChart()	Gráfico de empleados por estado con actualización de datos.
Chart.js	reportes.html · drawChart()	Visualización de duración agregada de los estados AUX.
SweetAlert2	js/gestion-datos.js · askConfirm(), notify()	Confirmaciones y retroalimentación del CRUD.
SheetJS	reportes.html · exportXLSX()	Libro Excel con detalle, resumen y filtros.
jsPDF + AutoTable	reportes.html · buildPDF()	Informe PDF con tablas y estadísticas.
JavaScript nativo	js/portal.js · appendAuxLog()	Registro de períodos AUX finalizados en ts_aux_logs.

5.1. Flujo de datos desde el empleado hasta el reporte

Cuando el empleado cambia de estado, el portal registra el período anterior —si existe una duración válida— en ts_aux_logs. Gestión también contempla el registro de transiciones administrativas. El módulo de Reportes consulta estos registros, los normaliza y aplica filtros antes de calcular métricas o exportar archivos. El dashboard obtiene los estados actuales desde ts_employees y actualiza sus indicadores y gráfica.

Origen	Proceso	Salida
Portal AUX / Gestión	Cierre y almacenamiento de períodos	ts_aux_logs
Gestión del personal	Lectura y edición de empleados	ts_employees
Dashboard	Conteos por colorId	KPIs y Chart.js
Reportes	Filtrar, agrupar y sumar duraciones	Tabla, CSV, XLSX y PDF

5.2. Validaciones avanzadas e interacción

El módulo de Gestión incorpora comprobaciones de campos obligatorios, correos con formato inválido y posibles duplicados mediante validateEmployee(). También utiliza validatePunch() para revisar formularios de marcación. Las confirmaciones y notificaciones acompañan esas reglas; de esta forma, una entrada inválida no se acepta solo porque el diálogo visual se haya mostrado.

5.3. Precauciones de exactitud

La duración se calcula a partir de fechas de inicio y finalización válidas. Los períodos sin duración positiva y los registros duplicados no deben incorporarse a las estadísticas. Los estados históricos ya retirados del selector se preservan como información heredada y no deben reasignarse sin fundamento a Disponible. El estado Salida representa el cierre de jornada, no un nuevo tiempo de actividad acumulable.

6. ANÁLISIS CRÍTICO DE LAS DECISIONES TÉCNICAS

Criterio	Decisión adoptada	Justificación / límite
Rapidez de incorporación	Librerías mediante CDN	Evita configuración compleja para un avance en HTML/JS; requiere conectividad.
Consistencia visual	Gráficas y KPI basados en el mismo estado	Permite contrastar totales y distribución en una sola pantalla.
Usabilidad	SweetAlert2 con validaciones nativas	Comunica acciones y reduce errores en el formulario.
Portabilidad de información	CSV, XLSX y PDF	Permite análisis posterior y envío de reportes a distintos destinatarios.
Persistencia	localStorage temporal	Útil para el prototipo; no sincroniza computadoras ni reemplaza un

		backend.
Seguridad	Sesiones simuladas y roles	Demuestra navegación diferenciada, pero no autoriza acceso seguro en producción.

7. CORRESPONDENCIA CON LA RÚBRICA DEL FORO 1

Requisito	Evidencia en el código	Evaluación documental
Gráfica funcional	renderAuxChart() y drawChart()	Se explica la generación y actualización con datos del proyecto.
Diálogos atractivos	SweetAlert2 en Gestión	Se describe el uso de confirmaciones, notificaciones y alternativas.
Módulo adicional	Reportes e historial AUX	Se documenta trazabilidad y exportación de períodos.
Validaciones avanzadas	validateEmployee() y validatePunch()	Se enlazan las reglas de formularios con la retroalimentación.
Investigación APA 7	Documento presente	Se presentan análisis propio, citas y fuentes técnicas.

Evidencia visual recomendada para la versión final: insertar capturas actuales del dashboard, un diálogo SweetAlert2 y la pantalla de Reportes con datos de prueba identificados como tales. Los mockups de la Fase 1 representan la propuesta inicial y no sustituyen capturas del software implementado.

8. CONCLUSIONES Y RECOMENDACIONES

8.1. Conclusiones

1. La incorporación de Chart.js, SweetAlert2, SheetJS y jsPDF amplía el valor del prototipo TimeShift al convertir registros operativos en información visual, verificable y exportable. Cada librería cumple una función delimitada y se vincula con un módulo concreto.

2. El mayor aporte técnico de esta etapa no es una gráfica aislada, sino la coordinación entre captura de cambios AUX, almacenamiento histórico, indicadores y reportes. Esa relación permite evaluar de forma consistente períodos de actividad y receso dentro de las limitaciones del almacenamiento local.

3. Separar HTML, estilos y lógica JavaScript en los módulos del dashboard facilita la comprensión y el mantenimiento. Las validaciones y los diálogos orientados al usuario aportan una experiencia más clara en las operaciones administrativas.

8.2. Recomendaciones

1. Migrar a una base de datos compartida con autenticación y reglas de autorización del lado del servidor antes de usar TimeShift en producción con datos reales de empleados.

2. Mantener pruebas sobre transiciones AUX, fechas inválidas, duplicados y coherencia entre tabla, KPIs y exportaciones.

3. Para el siguiente avance académico, evaluar la incorporación de React y una API externa según la ruta de aprendizaje, conservando las reglas de negocio comprobadas y evitando exponer credenciales en el navegador.

9. REFERENCIAS (APA 7.ª EDICIÓN)

Chart.js. (s. f.). Developers: API. https://www.chartjs.org/docs/latest/developers/api.html

Equipo TimeShift. (2026). Sistema de control de asistencia y gestión de turnos: Etapa 1, análisis y planificación del proyecto [Trabajo académico no publicado].

Parallax. (s. f.). jsPDF documentation. https://github.com/parallax/jsPDF

SheetJS. (s. f.). API reference: SheetJS Community Edition. https://docs.sheetjs.com/docs/api/

simonbengtsson. (s. f.). jsPDF-AutoTable [Repositorio de software]. GitHub. https://github.com/simonbengtsson/jsPDF-AutoTable

SweetAlert2. (s. f.). SweetAlert2 [Repositorio de software]. GitHub. https://github.com/sweetalert2/sweetalert2

TimeShift. (2026). TimeShift [Código fuente]. GitHub. https://github.com/Androx2000/TimeShift